

SMOHIT ARMAN SWAIN

192572092

CSA0802 PYTHON PROGRAMMING

CODING CHALLENGE

Q1 String Character Frequency Counter

10 marks

Write a Python program that accepts a string from the user and counts the frequency of each unique character (case-insensitive). Display the results in the format: character → count. Sort the output alphabetically by character.

Constraints:

- Ignore spaces in the count.
- Treat uppercase and lowercase as the same character.
- Sort output alphabetically.

Skills Assessed:

- String iteration, lower(), conditionals
- Dictionary or manual counting
- Sorting using loops or sorted()

Input: Hello World

output:

d → 1  
e → 1  
h → 1  
l → 3  
o → 2  
r → 1  
w → 1

1.

Your Code

```
1 s = input("Enter a string: ").lower()
2
3 freq = {}
4
5 for ch in s:
6     if ch != " ":
7         if ch in freq:
8             freq[ch] += 1
9         else:
10            freq[ch] = 1
11
12 for ch in sorted(freq):
13     print(ch, " " freq[ch])
```

✓ Submitted

Input

Hello World

Output

```
File
"C:\Windows\TEMP\sandbox_19257209
2RC_6a62409dba90c0.59777256\main.
py", line 13
    print(ch, " " freq[ch])
          ^^^^^^^^^^^^^
```

2.

### Q2 Vowel and Consonant Classifier with Statistics

10 marks

Accept a sentence from the user and display: (a) total vowels and consonants, (b) the most frequent vowel, (c) the least frequent consonant, and (d) the ratio of vowels to consonants as a decimal rounded to 2 places.

Constraints:

- Case-insensitive analysis.
- Ignore spaces and digits.
- Handle sentences with no vowels or no consonants gracefully.

Skills Assessed:

- Character classification with conditionals
- Counting and comparison logic
- Arithmetic and rounding

Sample Input :

Python Programming

Expected Output : Vowels: 5 | Consonants: 12 Most frequent vowel: o (2) Least frequent consonant: y (1) Vowel-to-Consonant Ratio: 0.42

```
1 import collections
2
3 def classify_text():
4     # Accept a sentence from the user
5     sentence = input("").strip()
6
7     # Define vowels and consonants
8     vowels_set = set('aeiou')
9     consonants_set = set('bcdfghjklmnpqrstvwxyz')
10
11     vowels_count = 0
12     consonants_count = 0
13
14     vowel_freq = collections.Counter()
15     consonant_freq = collections.Counter()
16
17     # Process each character (Case-insensitive analysis)
18     for char in sentence.lower():
19         if char in vowels_set:
20             vowels_count += 1
21             vowel_freq[char] += 1
22         elif char in consonants_set:
23             consonants_count += 1
24             consonant_freq[char] += 1
25
26     # Determine the most frequent vowel
```

python programming

Output

```
Vowels: 4 | Consonants: 14 | Most
frequent vowel: o (2) | Least
frequent consonant: t (1) | Vowel-
to-Consonant Ratio: 0.29
```

3.

### Q3 String Compression and Run-Length Encoding

10 marks

Implement a Run-Length Encoding (RLE) program that: (a) Compresses a string by replacing consecutive repeated characters with character+count. (b) Decompresses the encoded string back to the original. (c) If the compressed string is not shorter, returns the original.

Constraints:

- Single characters without count still output as single char.
- Handle empty strings.
- Decompressor should handle all valid RLE strings.

Skills Assessed:

- String traversal and comparison
- Count accumulation logic
- String building and integer conversion

Sample Input :

aabbbccdddee

Expected Output

Compressed: a2b3c2d4e2

Decompressed: aabbbccdddee

For input 'abc': Compression not beneficial.

Original:

#### Your Code

```

1 def compress_string(s: str) -> str:
2     # Handle empty strings
3     if not s:
4         return ""
5
6     compressed = []
7     n = len(s)
8     i = 0
9
10    while i < n:
11        char = s[i]
12        count = 0
13        # Count consecutive identical characters
14        while i < n and s[i] == char:
15            count += 1
16            i += 1
17
18        # Single characters without count still output as single char
19        if count == 1:
20            compressed.append(char)
21        else:
22            compressed.append(f"{char}{count}")
23
24    compressed_str = "".join(compressed)
25
26    # If the compressed string is not shorter, return the original
27    if len(compressed_str) >= len(s):

```

#### Input

aabbbccdddee

#### Output

```

--- Test Case 1 ---
Original:    aabbbccdddee
Compressed:  a2b3c2d3e2
Decompressed: aabbbddd
--- Test Case 2 ---

```

#### Output

```

--- Test Case 1 ---
Original:    aabbbccdddee
Compressed:  a2b3c2d3e2
Decompressed: aabbbddd

```

```

--- Test Case 2 ---

```

```

--- Test Case 2 ---
Original:    abc
Compressed:  abc (Compression
not beneficial)

```

Write a Python program that accepts a paragraph of text and computes: (a) Total words, sentences, and paragraphs. (b) Average word length and average sentence length. (c) The top 5 most frequent words (excluding common stop words: the, is, a, an, in, of, and, to, for). (d) Flesch Reading Ease Score:  $206.835 - 1.015 * (\text{words/sentences}) - 84.6 * (\text{syllables/words})$ . Approximate syllables as number of vowel groups per word.

Constraints:

- Sentence ends with . ! or ?
- Handle punctuation attached to words.
- Stop words list is hardcoded in the program.

Skills Assessed:

- Complex string parsing and tokenization
- Nested loops for syllable counting
- Floating-point arithmetic and rounding

Sample Input: Python is a powerful programming language

Expected Output :

Words: 45 | Sentences: 4 | Paragraphs: 1

Avg Word Length: 5.2 | Avg Sentence Length: 11.25

Top 5 Words: python(3), programming(2), language(2), data(2), code(1)

Flesch Reading Ease: 58.3 (Standard)

4.

```

25 )
26
27
28 common stop words
29 s", "a", "an", "in", "of", "and", "to", "for"}
30
31 (split by double newlines or single newlines if blocks)
32 rip() for p in text.split('\n') if p.strip()]
33 paragraphs_list)
34
35 (sentences end with ., !, or ?)
36 [.!?]+', text)
37 for s in sentences if s.strip()]
38 entences)
39
40 okenization (removing punctuation attached to words)
41 )
42
43 []
44
45
46
47
48 from edges of the word
49 ^[^\w]+| [^\w]+$', '', w)
50
51 .append(clean_w)
52 += count_syllables(clean_w)

```

## Input

Python is a powerful programming language

## Output

```
Sample Input: Python is a
powerful programming language.
Words: 6 | Sentences: 1 |
Paragraphs: 1
Avg Word Length: 6.0 | Avg
Sentence Length: 6.0
```

```
Sentence Length: 6.0
Top 5 Words: language(1),
powerful(1), programming(1),
python(1)
Flesch Reading Ease: 17.4 (Very
Confusing)
```